

Unidad 8:

Genéricos

Índice

1 Introducción. Tipos parametrizados o genéricos	3
2 Clases con parámetros genéricos	3
3 Interfaces con genéricos	5
4 Parámetros genéricos limitados	5
5 Métodos genéricos	7
6 Comodines	7
7 Cosas que no se pueden hacer con parámetros genéricos	8

1 Introducción. Tipos parametrizados o genéricos

El uso de los tipos genéricos obedece a la necesidad de disponer de clases, interfaces o métodos que se puedan usar con muchos tipos de datos distintos, pero haciendo comprobaciones de tipo en tiempo de compilación. Ejemplos importantes son los métodos de comparación, `compareTo()` y `compare()`, que aparecen en las interfaces `Comparable` y `Comparator` respectivamente. Ambas interfaces están pensadas para comparar objetos de cualquier clase. De hecho, tendremos que implementar `Comparable` para cualquier clase de objetos que insertemos en cualquier tabla o colección que pretendamos ordenar. Por eso el método `compareTo()`, antes de que aparecieran los genéricos con Java 5, recibía como parámetro una variable de tipo `Object`, que es la clase más general de todas. Si tenemos una tabla de objetos `Cliente` que queremos ordenar por su DNI, implementamos la interfaz `Comparable` en la clase `Cliente` basándonos en su atributo `dni`. La interfaz, cuyo único elemento es el método `compareTo()`.

2 Clases con parámetros genéricos

Supongamos que queremos definir la clase `Contenedor`, que permita guardar un solo objeto de cualquier tipo. Los únicos métodos serán `guardar()` y `extraer()`. Habrá un único atributo `objeto` que, en principio, podría ser de tipo `Object`, para que el objeto para guardar pueda ser de cualquier clase. Pero así no tenemos control sobre el tipo del objeto guardado. Desde luego, siempre podríamos implementar una clase `Contenedor` para `Integer`, otra para `Double` y así sucesivamente. Pero si lo que queremos es una clase `Contenedor` que sirva para todo tipo de objetos y que, a la vez, permita controlar en cada caso ese tipo, tenemos que recurrir a los tipos genéricos. Una clase `Contenedor` con tipo genérico `T` podría ser:

```

3 public class Contenedor<T> {
4     private T objeto; // se inicializa a null: contenedor vacío public Contenedor() {}
5
6     public void guardar(T nuevo) {
7         objeto = nuevo;
8     }
9
10    public T extraer() {
11        T res = objeto;
12        objeto = null; // el contenedor vuelve a estar vacío
13        return res;
14    }
15 }

```

`T` representa el tipo de datos que se va a usar en la clase en cada declaración concreta, y tiene que ser una clase o interfaz, nunca un tipo primitivo.

Ahora podemos crear un contenedor para enteros. La sintaxis es:

```
Contenedor<Integer> c = new Contenedor<Integer>();
```

El segundo `Integer` (el del lado derecho de la sentencia de asignación) puede ser omitido, ya que el compilador puede inferirlo a partir del lado izquierdo.

La expresión <> se llama operador diamante.

En este contenedor vamos a guardar un 5 y luego lo volvemos a extraer y lo mostramos por consola.

```
public static void main(String[] args) {
    Contenedor<Integer> c = new Contenedor<Integer>();
    c.guardar(5);
    Integer n = c.extraer();
    System.out.println(n); // aparece un 5 por consola
}
```

El compilador comprueba el tipo del valor que pasamos al método guardar(), que tiene que ser Integer. Si hubiéramos pasado el valor 7.4 o la cadena "silla", habría dado un error en la compilación. También hace una comprobación de tipos en la asignación a la variable n, que se ha declarado Integer. Si hubiéramos implementado la clase Contenedor con una variable object en vez de un tipo genérico, habríamos tenido que poner un cast Integer delante de c.extraer(), y si el objeto devuelto fuera de tipo distinto a Integer, el error se habría producido durante la ejecución del programa.

La misma clase nos sirve para crear un contenedor de números reales:

```
Contenedor <Double> c1 = new Contenedor<>();
```

o Clientes:

```
Contenedor<Cliente> c2 = new Contenedor<>();
```

En general, para definir una clase con un tipo genérico T, se escribe <T> después del nombre de la clase.

```
class NombreClase<T> {
}
```

Se suele usar la letra T para el tipo genérico, pero puede ser cualquier otra, aunque es costumbre reservar B para elementos de colecciones, K para claves, V para valores o N para números.

En la implementación de una clase puede intervenir más de un tipo genérico U, V... En ese caso, se especifican los parámetros separados por comas.

```
class nombreClase <U, V...> {
}
```

Las clases definidas con tipos genéricos, como nuestro contenedor, también pueden usarse sin ellos, en cuyo caso el compilador trabaja por defecto con variables de tipo object. Eso significa que no hace comprobaciones de tipos y se pueden guardar objetos de distintas clases mezclados. Podemos escribir:

```
Contenedor c = new Contenedor();
c.guardar(7); // el atributo objeto guarda un 7 como object
c.guardar("roca"); // ahora guarda la cadena "roca" como object
Double x = (Double) c.extraer ();
```

El compilador se limita a darnos un aviso de que estamos realizando operaciones sin comprobación de tipo. El error se producirá al ejecutar el programa, con una excepción `ClassCastException` al aplicar el cast.

Muchos métodos de clases o interfaces de `java.lang`, como `compareTo()` o `compare()`, que actualmente están implementados con tipos genéricos, siguen soportando la implementación antigua con parámetros `object`, aunque debe evitarse cuando sea posible, ya que eso sería renunciar a la ventaja de los tipos genéricos, que es la comprobación en tiempo de compilación de los tipos de los datos pasados como parámetro a las funciones o asignados a las variables.

3 Interfaces con genéricos

También se pueden definir interfaces con tipos genéricos y la sintaxis es idéntica,

```
interface NombreInterfaz<T> {
}
```

Un ejemplo de interfaz con tipo genérico, definida de esta forma desde Java 5 y presente en `java.lang`, es la interfaz `Comparable`.

```
public interface Comparable<T> {
}
int compareTo(TO);
```

Si queremos implementarla en la clase `Cliente` para que tenga un orden natural basado en el DNI, escribiremos,

```
3 public class Cliente implements Comparable<Cliente> {
4     private String dni;
5     private String nombre;
6
7     public int compareTo(Cliente o) {
8
9         return dni.compareTo(o.dni);
10    }
11 }
```

4 Parámetros genéricos limitados

La implementación de una clase con el tipo genérico `T` implica que, en sus métodos, se van a realizar operaciones con variables de dicho tipo. Pero, a veces, dichas operaciones solo tienen sentido para determinados tipos de `T`. Por ejemplo, si aparece alguna operación aritmética entre valores de tipo `T`, sabemos que este no puede ser `String` o `Cliente`. Para casos así, existen los tipos genéricos limitados. La idea es que se limiten los

posibles tipos de T a una determinada clase claseLimite y todas sus subclases (si claseLimite es un límite superior) o todas sus superclases (si claseLimite es un límite inferior). En el primer caso,

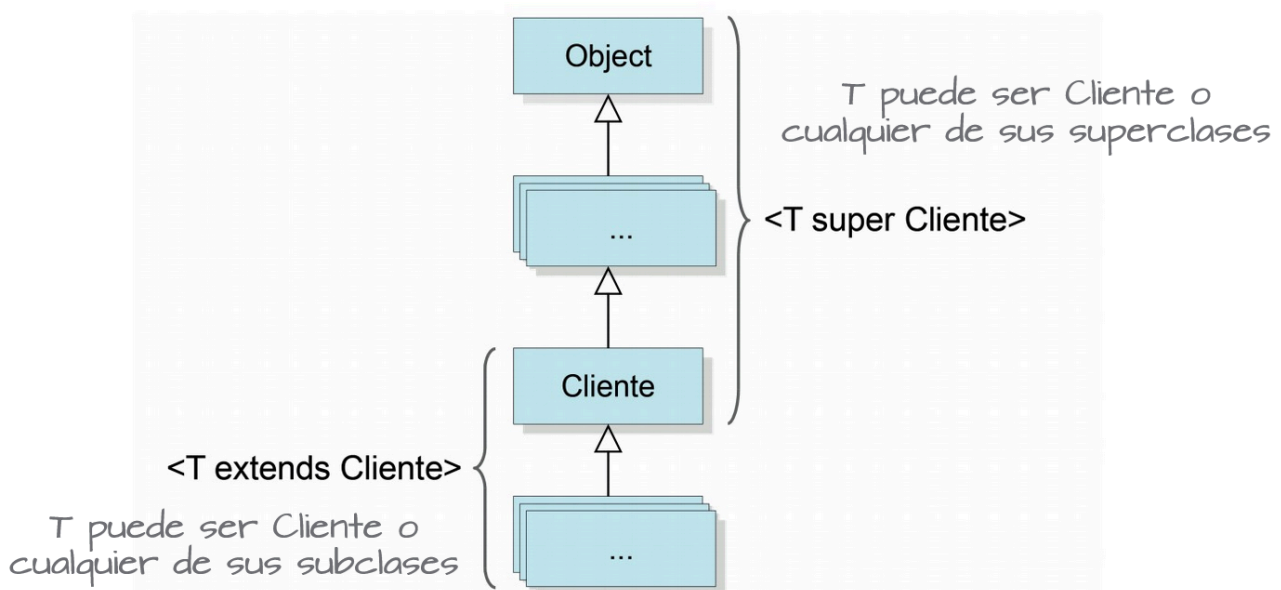
```
class NombreClase<T extends claseLimite> {
}
```

<T extends claseLimite> significa que T puede ser claseLimite o cualquiera de sus subclases.

O bien

```
class NombreClase<T super claseLimite> {
}
```

<T super claseLimite> significa claseLimite y todas sus superclases.



Como ejemplo, supongamos que queremos implementar la clase Calculadora con el tipo genérico T, donde se realizarán operaciones aritméticas. En este caso debemos limitarnos a las clases envoltorio que heredan de la clase abstracta Number (Integer, Double...). La clase Calculadora sería de la forma:

```
3 public class Calculadora<T extends Number> {
4
5     private T a, b;
6
7     //operaciones con a y b
8 }
```

Cuando declaremos una variable o usemos el constructor de una clase con tipo genérico, el compilador comprobará si el tipo declarado está dentro de los límites del parámetro genérico que aparece en la definición de la clase. Por ejemplo, la sentencia

Calculadora<Double> c = new Calculadora<>();
será correcta, ya que Double es una subclase de Number.

En cambio, se producirá un error de compilación con

```
Calculadora<Object> c = new Calculadora<>();
ya que object no hereda de Number.
```

También se pueden limitar los tipos genéricos a aquellos que implementan una o más interfaces. En este caso no se usa la palabra `implements`, sino `extends`, como si fuera una herencia. Esta es una particularidad exclusiva de la sintaxis de los parámetros genéricos. Por ejemplo, si estamos definiendo una clase `MiClase` con un parámetro genérico `T`, que sea válido solo para valores que tengan implementada la interfaz `MiInterfaz`, escribi remos:

```
class MiClase<T extends MiInterfaz> { //¡no ponemos implements!
}
```

5 Métodos genéricos

Los parámetros genéricos de una clase o interfaz suelen aparecer en los métodos implementados dentro de ella. Por ejemplo, en los métodos `guardar()` y `extraer()` de la clase `Contenedor`, aparece el parámetro `T`. Sin embargo, dentro de cualquier clase, tanto si está definida con tipos genéricos como si no, podemos implementar métodos con sus propios parámetros genéricos, distintos de los que pueda tener la clase. Dichos métodos se llaman métodos genéricos.

Como ejemplo, vamos a implementar un método que nos devuelve el número de elementos `null` que hay en una tabla que se le pasa como argumento. El tipo `U` de la tabla es genérico, y se declara en la definición del método, justo antes del tipo devuelto.

```
public static <U> int numeroDeNulos(U[] tabla) {
    int cont = 0;
    for (U e : tabla) {
        if (e == null) {
            cont++;
        }
    }
    return cont;
}
```

Este método se puede incluir en cualquier clase y no depende para nada de los parámetros propios de ella.

El tipo asociado a un método genérico también puede estar limitado. Por ejemplo, si quisiéramos que nuestro método `numeroDeNulos()` solo funcionara para tablas numéricas, pondríamos `<U extends Number>` en vez de `<U>`.

6 Comodines

Los comodines -o wildcards- se suelen usar en la declaración de atributos, variables locales o parámetros pasados a una función. Un comodín se representa con el símbolo “?”, que significa cualquier tipo. Por ejemplo, si escribimos

```
Contenedor<?> c;
```

hemos declarado una variable `c` de tipo `Contenedor` cuyo parámetro genérico asociado puede ser cualquiera. La variable `c` puede referenciar un `Contenedor` de `Integer`, de `String` o de cualquier otra clase. Esto significa que todos los objetos `Contenedor` pertenecen a alguna subclase de `Contenedor<?>`.

Aquí tenemos que llamar la atención sobre un error común cuando se usan los tipos genéricos. El hecho de que `Integer` sea subclase de `Number` no implica que `Contenedor<Integer>` sea subclase de `Contenedor<Number>`. De igual modo `Contenedor<Cliente>` no es subclase de `Contenedor<Object>`. La relación de herencia entre los valores del parámetro genérico no tiene nada que ver con la relación entre las instancias de la clase `Contenedor`.

Por ejemplo, la sentencia

```
Contenedor<Object> cobj = new Contenedor<Integer>(); //!Error!
```

producirá un error de compilación, a pesar de que `Integer` hereda de `Object`. Si queremos una variable `c` de tipo `Contenedor` que pueda referenciar a cualquier objeto `Contenedor`, tendremos que usar `Contenedor<?> c`.

La relación de herencia entre clases usadas como tipos genéricos en otras clases (como las colecciones) no implica que estas últimas mantengan la misma relación de herencia.

Con el comodín también se puede usar la clase límite superior `<? extends T>`, que significa cualquier clase que herede de `T`, incluyendo a esta. E inferior `<? super T>`, que significa `T` o cualquier superclase de `T`.

Cuando escribimos

```
Contenedor<? extends Number> c;
```

estamos declarando una variable de tipo `Contenedor` de clase numérica. Puede referenciar un objeto `Contenedor<Number>`, `Contenedor<Integer>`, `Contenedor<Double>`, etc. Todas ellas son subclases de `Contenedor<? extends Number>`

Ejemplo:

```
public static double sumar(ArrayList<? extends Number> lista) {
    double total = 0;
    for (Number elemento : lista) {
        total += elemento.doubleValue();
    }
    return total;
}
```

7 Cosas que no se pueden hacer con parámetros genéricos

A pesar de todas las aportaciones de las clases genéricas a la programación en java, por la forma en que estas han sido implementadas, tienen una serie de limitaciones, algunas de las cuales puede que sean subsanadas en el futuro. En concreto, hay varias operaciones que, de momento, están prohibidas. Las más importantes son:

Los tipos genéricos nunca pueden ser primitivos. Cuando estos hagan falta, usaremos sus correspondientes clases envoltorio Integer, Character...

No se pueden crear instancias de tipo genérico, como en `new T();`

No se pueden crear tablas de tipos genéricos, como en `new T[10]`. Cuando hagan falta para un tipo concreto, se pasan como argumento al método donde van a ser usadas, que puede ser un constructor, o deberán ser devueltos por algún método definido fuera de la clase. Las tablas genéricas siempre tienen que venir construidas fuera de nuestra clase, interfaz o método genérico.

Tampoco se pueden crear tablas de clases parametrizadas, como `new Contenedor<Integer> [5];`

No se pueden usar excepciones genéricas.